One highlight, from capture to report

Demo walkthrough without Vergil

Current commands and reviewer controls | October 26, 2026

An eight-minute presentation using one captured TEST ONLY submission, local review, explicit approval and a local PDF. The screenshot appendix preserves dated cloud evidence.

01 // Capture

Choose SharePoint receipt/file or GitLab immutable commit

02 // Review and apply

Inspect one proposed entry, then confirm the local change

03 // Graduate

Select the contributor promotion and confirm it separately

04 // Report

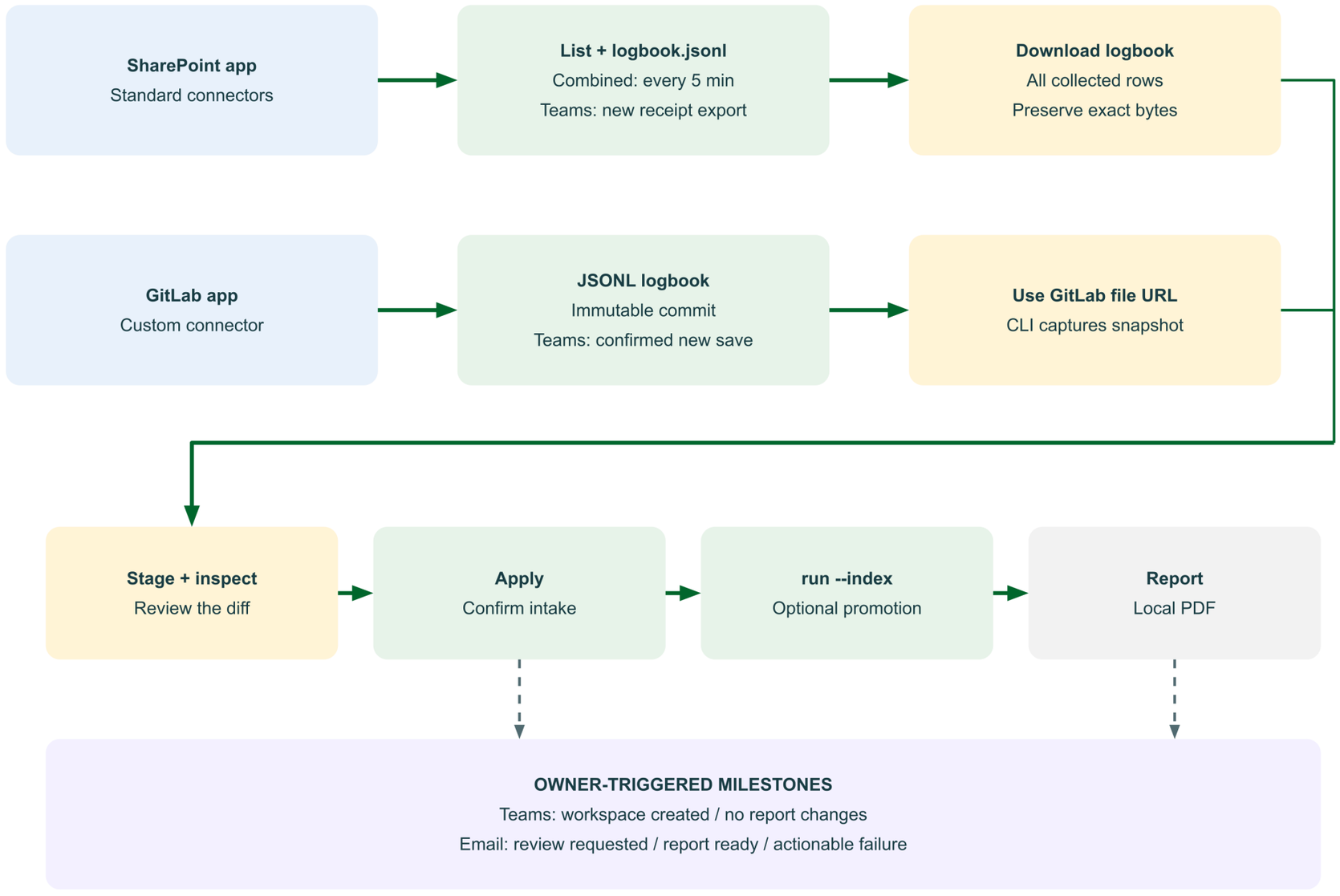
Open the local PDF and demonstrate safe replay

The following map shows both capture branches. New-save Teams notices are automatic and best effort. Later review/report messages use the owner-triggered callback. The retained full rehearsal uses the GitLab fixture.

WORKFLOW MAP

From highlight to reviewed report

Choose one intake route per highlight. Both preserve the original submission for later review.



Solid green: data handoff. Teams notices at save are automatic and best effort. Dashed: owner-triggered milestones; the local CLI sends no messages. A local PDF does not itself publish a report or send email.

Demo scope and evidence

Plan assets now live in local `_assets/` folders. The demo walkthrough PDF was refreshed October 26, 2026 with both intake branches, current commands and confirmation controls. Historical screenshots and execution records retain their original paths; the links and commands in this walkthrough use the current layout.

This packet explains both capture routes. Its retained end-to-end rehearsal follows **one actual GitLab PowerApp submission** through JSONL, local review, ingestion, contributor graduation and a local PDF. SharePoint receipt/export and Teams tests passed separately; the downloaded SharePoint file-to-CLI handoff has not yet been accepted. The contributor, baseline project and report are **TEST ONLY**. Nothing in the local rehearsal publishes a report, sends a callback, drains the remote inbox or starts Vergil.

Start here

• Presentation walkthrough: the eight-minute story and captured screens.

• Resulting report: the archived personnel entry remains visible in the fixed October 24 reporting window.

• Rehearsal evidence: source and output hashes, local Git revision, exact commands, exit codes and elapsed time.

• Failure acceptance: isolated, non-writing test of the single unresolved response.

• CLI guide: supported command contract.

Eight-minute presentation

Time	Role and action	Point to show
0:00-1:30	Contributor opens the assigned SharePoint or GitLab app, loads projects and enters a non-sensitive highlight.	Use the recorded prepared form and saved receipt if cloud access fails. The saved example is category Personnel, status Completed, summary `Example, Avery (TEST ONLY)`.
1:30-2:15	Operator shows the branch-specific receipt and JSONL source.	SharePoint: list receipt plus combined `logbook.jsonl`, refreshed every five minutes. Individual receipt files remain evidence. GitLab: immutable commit with one added line. Neither approves content. The retained local rehearsal uses the GitLab personnel fixture.
2:15-3:30	Reviewer runs stage and inspect.	Explicit project/member mapping, exact source bytes, one proposed entry and a readable diff. The index is unchanged before approval.
3:30-4:15	Reviewer approves the disposable local change and applies it.	One entry. Repeat apply produces no duplicate.
4:15-5:30	Reviewer previews graduation candidates, then selects the contributor entry with the reviewed digest.	Ingestion and graduation are separate approvals. Graduation adds one contributor and archives the retained source entry; unrelated metadata stays unchanged.
5:30-6:30	Operator opens the local PDF.	The same entry is visible with its Archived label. The output is local, not a published sponsor report.
6:30-7:30	Operator repeats graduation and tries an unknown project.	Stale graduation and rejected input fail without changing the index.
7:30-8:00	Presenter explains the next step.	Vergil will automate intake/review plumbing. Canonical review and promotion remain explicit. Shared deployment and production access approval are separate work.

The automated eleven-step rehearsal completed in 2.01 seconds on October 24, 2026, excluding human explanation and approval time. The eight minutes above is a presentation allocation, not a measured human rehearsal.

Choose the capture branch

Branch	Capture and storage	Handoff to local ACORN
SharePoint pilot	Standard SharePoint intake/catalog connections; one list receipt and one UUID-named receipt file. A separate five-minute exporter collects all rows into logbook.jsonl.	Download combined logbook.jsonl after its next refresh and stage its local path or file URI. Preserve exact bytes; list CSV/Excel export is not the intake envelope.
GitLab connector app	Existing custom connector; append to project 22464/main/.acorn/intake/logbook.jsonl. Confirmation identifies an immutable commit.	Stage a GitLab raw/blob file URL; the CLI captures an immutable snapshot. An already captured local file also works.

The SharePoint catalog contains 13 seeded projects; it is not automatically synchronized with GitLab. Owner-context app submission, exact export, reload recovery and Teams action tests passed on October 26. Standard-license participant access, download-to-CLI acceptance, failure recovery and broader boundary/concurrency tests remain open. The original GitLab app remains operational; intermittent GitLab access is a known limitation. Use one branch per highlight. Do not reroute an unresolved receipt into the other app.

Example handoff commands, using an approved ACRE configuration with local destination indexes and member rules:

```
# SharePoint: download combined logbook.jsonl after the next five-minute refresh.
acorn intake stage --batch --config intake.json \
  --source "/path/to/logbook.jsonl"

# GitLab: resolve and save an exact source snapshot.
CI_SERVER_HOST=code.ornl.gov acorn intake stage --batch \
  --config intake.json --source "<GitLab raw/blob file URL>"

# Both routes converge here; use your source's batch ID.
acorn intake inspect --config intake.json --batch-id <batch-ID>
acorn intake apply --config intake.json --batch-id <batch-ID>
```

Direct SharePoint HTTPS sources are not supported by the CLI source adapter. Do not regenerate the envelope, UUID or JSON formatting. These commands describe the SharePoint handoff to validate; they do not claim it has been rehearsed. The frozen demo script below continues to use its verified GitLab fixture.

Where notifications occur

Both intake branches send best-effort Teams notices to the owner after a confirmed new save. SharePoint requires a new receipt and a verified export in the same run; if export first succeeds on a later retry, the initial notice is skipped. GitLab requires the confirmed new-commit response. Exact replays do not send another notice. There is no routine intake email.

Later milestones use the separate owner-triggered callback: Teams for workspace created or no report changes; email for review requested, report ready or actionable failure. The local CLI does not trigger those messages. A local PDF is not evidence of remote publication. The page 2 diagram shows both routes and the manual milestone boundary. See notification routing and SharePoint acceptance.

Repeat safely

Build the current CLI (`cargo build --bin acorn`, default PDF feature), then run from the repository root:

```
python3 .wiki/features/manual-jsonl-ingestion-graduation/_assets/rehearse.py \
  --workspace /tmp/acorn-demo-next-week-01 \
  --acorn target/debug/acorn \
  --chrome '/Applications/Google Chrome.app/Contents/MacOS/Google Chrome'
```

The workspace must not exist. The script refuses to reset an existing directory. Use a new name for each rehearsal; retain earlier evidence. It uses ACORN's **Yes / No** buttons for intake approval, selection of the contributor promotion, and final graduation approval. No is selected by default; Tab or the arrow keys switches selection and Enter confirms it. `y` and `n` also work; Escape or Ctrl+C cancels. No skips a graduation candidate; in this one-candidate rehearsal, skipping it stops the expected graduation checks. Declining the intake or final

graduation approval stops the rehearsal without applying that change. Run in a terminal with both input and output attached. Interactive receipts appear in the terminal; the evidence records each command, confirmation mode and exit status.

`--approve-synthetic` permits an unattended check of **only the frozen TEST ONLY fixture**. It supplies `--yes` for intake apply and explicit `--accept` plus the preview digest for graduation. This is not Jason's personal review of production data. Apply replay and rejection checks also pass `--yes` to test validation without additional prompts. `--git /absolute/path/to/git` can select a Git executable if the system toolchain is unavailable.

The current graduation API is:

```
acorn intake run --index project/index.json --dry-run
acorn intake run --index project/index.json --interactive
# Accept all eligible promotions without prompting:
acorn intake run --index project/index.json --force
```

`--force` preserves validation, stale-digest checks and exact backups. Combining it with `--dry-run` writes nothing. The older `candidates` and `graduate` commands visible in historical transcripts still work as hidden compatibility commands; the refreshed PDFs use `run`. See the current command reference for reviewed-ID selection and all mode combinations.

The script verifies the immutable source hashes and exact line selection before writing, creates a disposable index, records all command results, creates a local Git commit using the explicitly synthetic `ACORN Synthetic Demo` identity, and renders the PDF twice. It asserts that retries, rejected apply and both report runs preserve the correct index. `evidence.json` is the local report manifest; `remotePublication` is false.

Confirmation acceptance

The earlier October 24 confirmation update passed 20 terminal checks and three interactive rehearsal paths: No, default Enter, Escape and Ctrl+C preserved the local index; Yes applied the reviewed change. Declining graduation also left no backup file. Revoking folder membership while either Apply prompt was open prevented the write after Yes, confirming that permissions are reloaded after approval. The complete interactive rehearsal passed all eleven steps with two native confirmation prompts; its recorded evidence records the confirmation mode for each command. The unattended synthetic rehearsal also passed using `--yes`. CLI integration tests verify redirected input/output cannot approve implicitly, and `--yes` retains stale-index, digest and replay protections. Final validation passed: `make lint`, `make test` (2,672 passed; 17 skipped), and the planning wiki validator.

The consolidated `run` API passed ten native terminal checks: per-entry selection, default No, final rejection, Escape/Ctrl+C, interactive dry-run, exact backups, and rejection of concurrent index changes. Both the unattended rehearsal and interactive rehearsal passed all eleven steps with the current commands. The interactive rehearsal used three native prompts. These checks used synthetic local data and sent no messages or repository updates. Final validation for the consolidated API passed: warning-free `make lint`, `make test` (2,694 passed; 17 skipped), the documentation book build, and the planning wiki validator.

The original rehearsal manifest and screenshots remain the captured demonstration record. This walkthrough and its refreshed PDF describe the current button controls and unattended invocation contract. The PDF adds captured screens and the workflow notification map.

Exact trace

Evidence	Value
GitLab project / branch	`22464` / `main`
Inbox	`.acorn/intake/logbook.jsonl`
Submission UUID	`4313fcfc-0201-441e-8bda-c77718a31dcf`
Immutable source commit	`3090aa5714b4c6ff59c5dd0b6ecba999f918caa2`
Source parent	`7dd33b6d6094ab4d0b2ce13d8c58296a210064d5`
Append run	`085840872304453944518423985555CU19`
Exact replay run	`085840872270677292120848367 66CU19`
Local entry	`form-4313-fcfc-0201-441-e-8-bda-c-77718-a-31-dcf`
Selected source	Line 6 of six lines; complete original bytes including LF
Snapshot SHA-256	`76f0e36f637b5611cca8aa6c69ce5db13be00a0b5e17b2dbfe71c20a38177384`
Selected line SHA-256	`c8496e27ce6379296d19c194305b2f17f028f2e3314230de9ce8adc7b28a2c58`
Reporting window	`2026-10-24T00:00:00Z` inclusive to `2026-10-25T00:00:00Z` exclusive

The replay run reached **Queued replay** after reading the prior immutable commit and its parent; the new-write branch was skipped. Response values remain protected in run history. The saved flow's replay response references that verified prior commit. The original app receipt and the downloaded snapshot establish the UUID/commit mapping.

`_assets/baseline.json` is a separate synthetic local project, not a downloaded canonical project index. `_assets/selected.jsonl` is unchanged capture data. `_assets/rejected.jsonl` intentionally changes the project selector and is a separate negative fixture, never submitted to the live flow. The index's contributor and selected archive flag are the only semantic changes between `_assets/ingested.json` and `_assets/graduated.json`.

Day-of fallback and boundaries

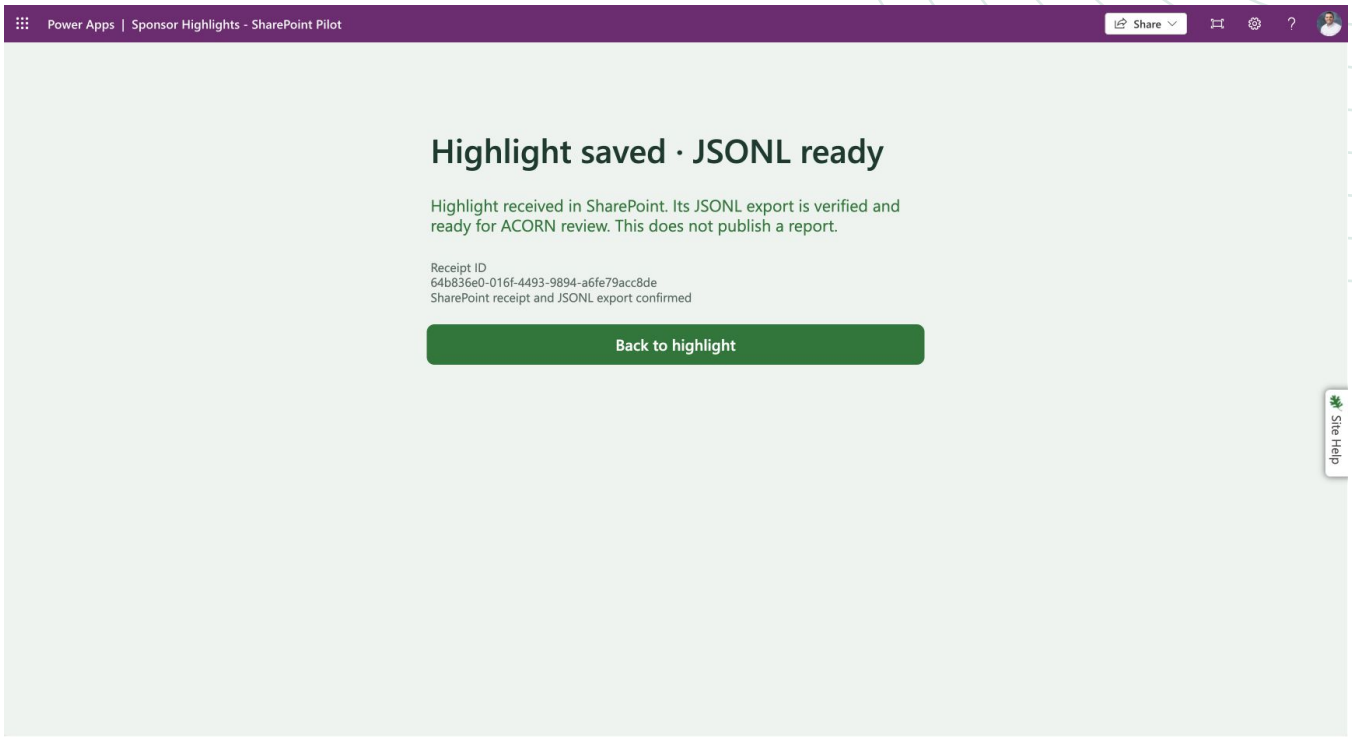
Try the approved cloud route once. If access is denied, stop that segment and show the captured form receipt, commit and replay screenshots; continue the local script with the exact frozen source. Say **“recorded cloud capture, live local review and reporting.”** Do not claim that a new cloud submission succeeded.

Use one browser tab and non-sensitive examples. A network denial is not a reason to rotate credentials or weaken access controls. Reliable egress/licensing approval and wider browser/concurrency acceptance remain in the intake plan. The remote TEST ONLY records remain in place. Before enabling a bot against any manually processed production data, approve the actual baseline using the verified handback procedure; local receipts alone do not prevent a fresh bot database from admitting those lines again.

The demonstration uses `report --dry-run --combine` and an explicit local-only evidence manifest. This avoids presenting publication-shaped previews as evidence of remote publication.

SharePoint: verified receipt and export

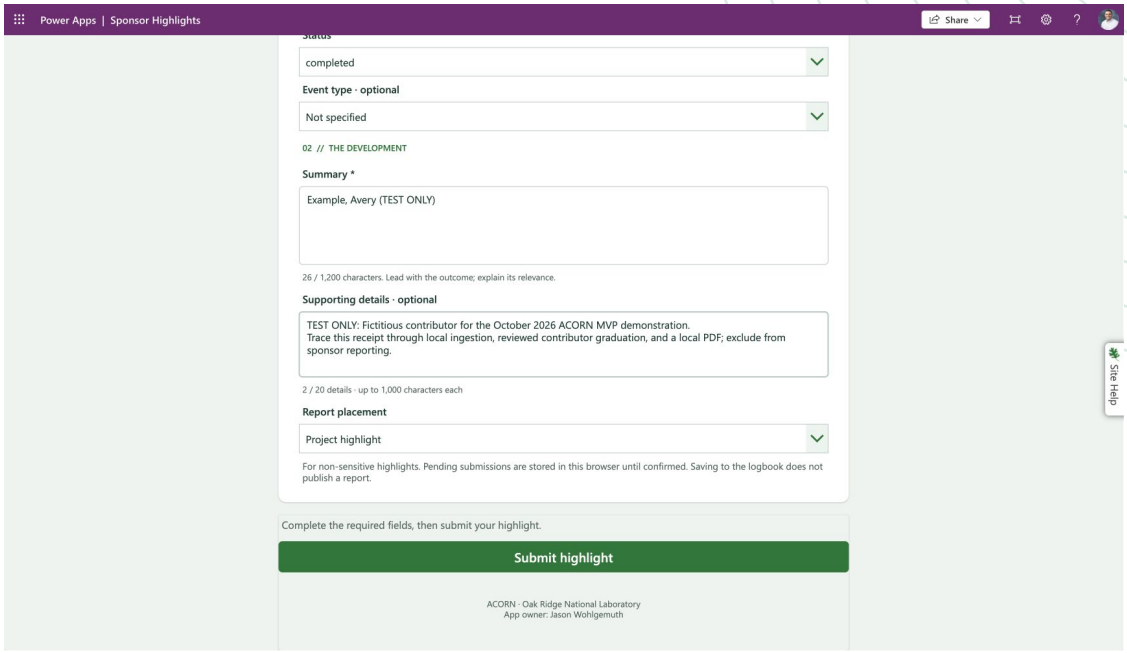
October 26, 2026: owner-context published app created a list receipt and verified its JSONL export. The Teams action succeeded. This is intake evidence; the downloaded-file-to-CLI handoff and standard-license participant access remain unaccepted.



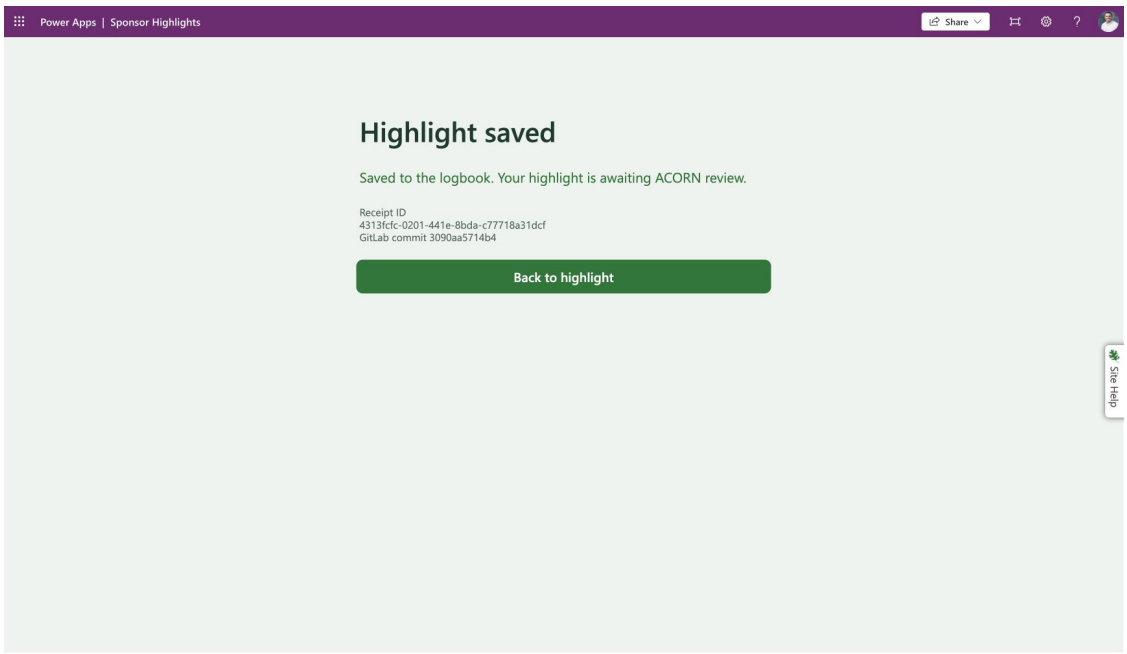
Receipt 64b836e0-016f-4493-9894-a6fe79acc8de. One file per receipt; no GitLab append is performed by this branch.

GitLab: captured form and receipt

Historical evidence captured October 24, 2026. UI labels or paths in these screens may differ from current commands.



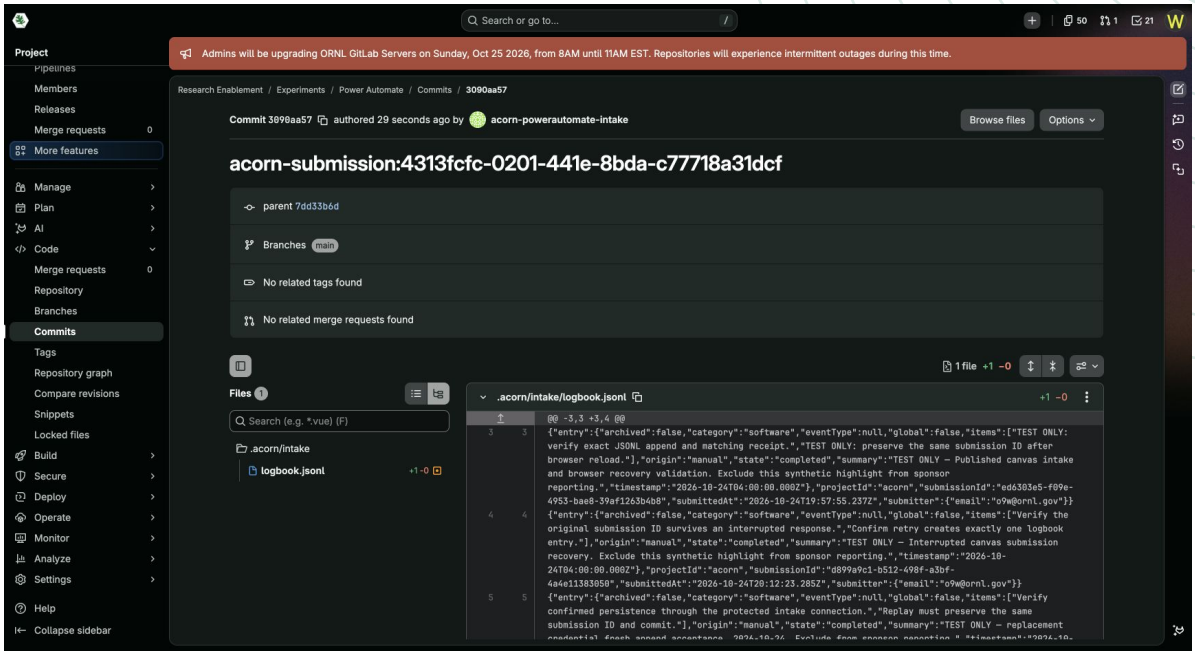
Prepared TEST ONLY personnel highlight.



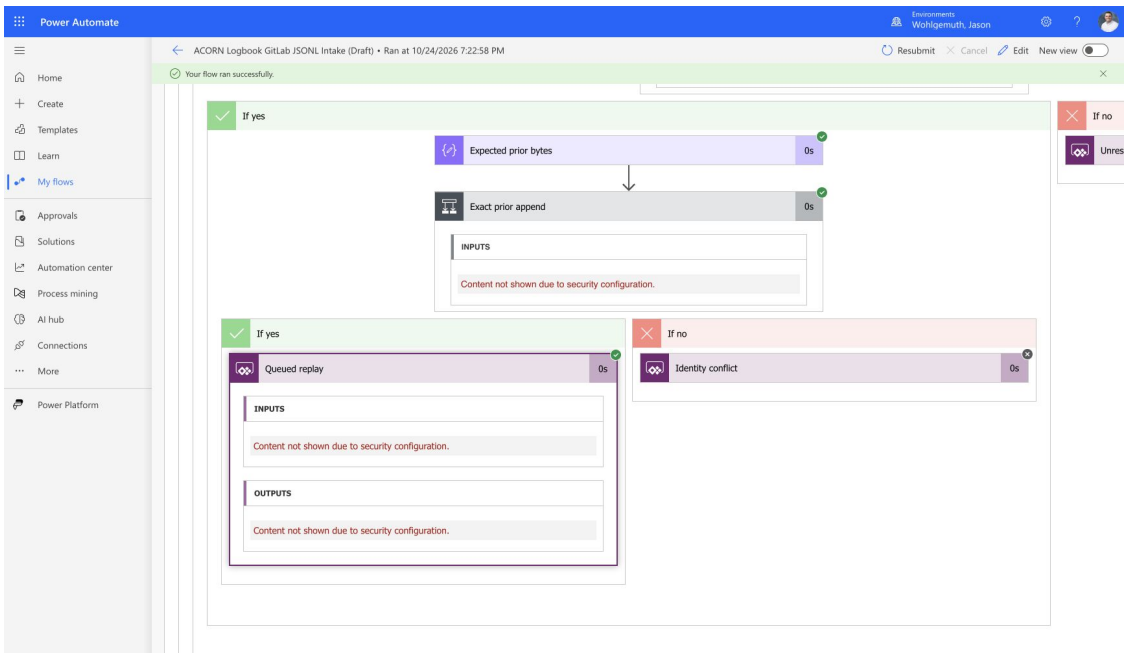
Saved receipt: capture is confirmed; content review follows.

Captured append and replay

Historical evidence captured October 24, 2026. UI labels or paths in these screens may differ from current commands.



GitLab commit: exactly one added JSONL line.



Exact replay: the prior immutable commit is reused.

Local report result

Historical evidence captured October 24, 2026. UI labels or paths in these screens may differ from current commands.



Local TEST ONLY report. The retained entry is archived; this is not remote publication.